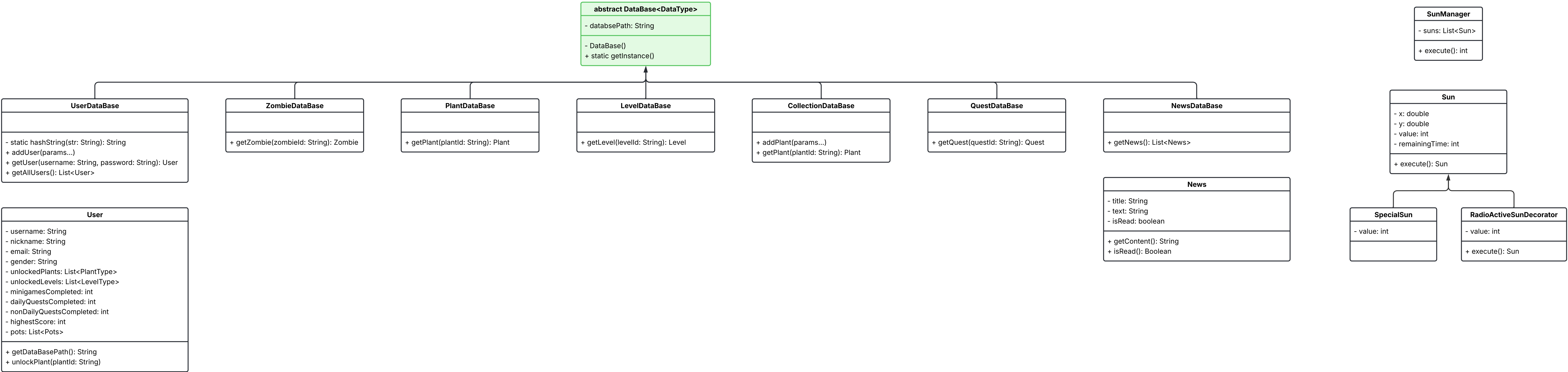




GameSession
- player: Player - level: Level
+ play(levelId: String) + advanceTime(ticks: int) + collectSun(x: int, y: int) + addSun(count: int) + releaseTheNuke() + plantPlant(type: String, x: int, y: int): String + removeCooldown(): String + pluckPlant(x: int, y: int): String + feedPlant(x: int, y: int): String + plantFood(): String + startZombieWaves(): String + zombieInfo(): String + spawnZombie(type: String, x: int, y: int): String

Chapter
- name: String - levels: List<Level> - isUnlocked: boolean - sideModifiers: List<SideModifiers>
+ LevellsPlayable(level: levelId): boolean

Level
- player: Player - levelId: String - name: String - gameBoard: GameBoard - waveManager: WaveManager - economyManager: EconomyManager - winCondition: WinCondition - lossCondition: LossCondition - sideModifiers: List<SideModifiers> - sun: int - zombies: List<Zombie>
- getPlants(): List<Plant> - shouldSpawnNewWave(): boolean - checkWinCondition(): boolean - checkLossCondition(): boolean - applySideModifiers() + run(): boolean + getAllowedPlants(): List<Plant>

GameBoardBuilder
game board parameters
+ buildGameBoard(levelId: String): GameBoard

GameBoard
- lanes: List<Lane> - totalRows: int - totalCols: int
+ getTileAt(x: int, y: int): Tile

Lane
- tiles: List<Tile> - lawnMower: LawnMower
+ getEntities(): List<Entity> + getPlants(): List<Plant> + getZombies(): List<Zombie>

Tile
- type: TileType - plant: Plant - obstacle: Obstacle
+ isPlantable(): boolean + tick(level: Level)

LawnMower
- lane: Lane - isActivated: boolean
+ execute()

GameTime
- ticks: int
+ advanceTime(ticks: int) - getTime(): int

enum WaveManager
STANDARD NO_WAVES
- waveCounter: int - waveZombies: List<Zombie> - delay: int - lastTime: int
+ tick(level: Level)

enum EconomyManager
STANDARD CONVEYOR_BELT FIXED_SUN NIGHT
+ tick(level: Level): int

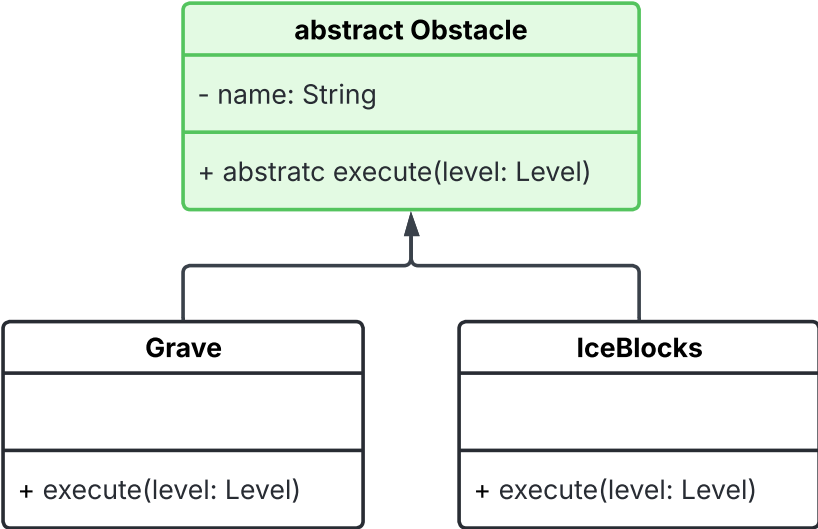
enum WinCondition
STANDARD TIMED_WAR
+ checkCondition(level: Level): boolean

enum LossCondition
STANDARD SAVE_OUR_SEEDS DEAD_LINE LOVE_YOUR_PLANTS
+ checkCondition(level: Level): boolean

enum SideModifier
STANDARD CONVEYOR_BELT FIXED_SUN NIGHT
+ apply(level: Level)

enum TileType
ANCIENT_EGYPT FROSTBITE_CAVE SPLIPPERY BIG_WAVE_BEACH DEEP_BEACH DARK_AGES NECROMANCY SAVE_SEED
- name: String
+ getByName(name: String): TerrainType + tick(level: Level)

enum LevelType
STANDARD_ANCIENT_EGYPT STANDARD_FROSTBITE_CAVE STANDARD_BIG_WAVE_BEACH STANDARD_DARK_AGES CONVEYOR_BELT LOCKED_PLANTS SAVE_OUR_SEEDS TIMED_WAR NIGHT_OPS DEAD_LINE LOVE_YOUR_PLANTS PLANT_WHAT_YOU_GET
- name: String
+ getByName(name: String): LevelType + execute(level: Level)



This represents one row in the leaderboard table. One instance per registered player.

LeaderboardEntry — concrete

- username: String

← pulled from Player when entry is built

- lastChapter: String

← name of last chapter the player completed e.g. "Ancient Egypt"

- lastLevel: int

← level number within that chapter e.g. 3

- minigamesCompleted: int

← total count of minigames player has finished successfully

- dailyQuestsCompleted: int

← count of daily quests completed tracked separately from non-daily

- nonDailyQuestsCompleted: int

← count of non-daily quests completed

- highScore: int

← highest myupoint score ever achieved in scored game mode

LeaderboardEntry
-username: String -lastChapter: String -lastLevel: int -minigamesCompleted: int -dailyQuestsCompleted: int -nonDailyQuestsCompleted: int -highScore: int

Relationship to Player

Leaderboard needs Player passed into buildEntries() because all the stats live on Player.

The fields you need from Player:

Player (partial — only what Leaderboard touches)

+ getUsername(): String

+ getLastChapter(): String

+ getLastLevel(): int

+ getMinigamesCompleted(): int

+ getDailyQuestsCompleted(): int

+ getNonDailyQuestsCompleted(): int

+ getHighScore(): int

- entries: List<LeaderboardEntry>

← one entry per registered player

← built from all Player objects when leaderboard is opened

← LeaderboardMenuController calls buildEntries() when player navigates to leaderboard menu

+ buildEntries(players: List<Player>): void

← iterates all registered players creates a LeaderboardEntry for each one from their current stats stores in entries list

← called by LeaderboardMenuController when menu is entered so data is always fresh

+ sortBy(column: LeaderboardColumn ascending: boolean): void

← sorts entries list by the specified column

← ascending true = lowest to highest false = highest to lowest

← LeaderboardMenuController calls this when player requests sorting

← LeaderboardColumn is an enum so invalid column names are impossible

+ show(): String

← returns text representation of sorted entries table

← called by LeaderboardMenuController after sorting to display result

leaderboard
-entries: List<LeaderboardEntry>
+buildEntries(players: List<Player>): void +sortBy(column: LeaderboardColumn acending: boolean): void +show(): String

public enum LeaderboardColumn
USERNAME LAST_LEVEL MINIGAMES_COMPLETED DAILY_QUESTS_COMPLETED NON_DAILY_QUESTS_COMPLETED HIGH_SCORE

GreenHouse
- player: Player - pots: List<Pot>
+ getPot(x: int, y: int): Pot + plantPot(x: int, y: int) + collectPot(x: int, y: int) + accelerateGrowth(x: int, y: int) + showGreenHouse(): String

Pot
- x: int - y: int - locked: boolean - plantedPlot: PlantTemplate - plantedAt: LocalDateTime - lastHarvestTime: LocalDateTime - growthDurationHours: int - isMarigold: boolean
+ isLocked(): boolean + isEmpty(): boolean + isReady(): boolean + getHoursRemaining(): double + unlock() + grow(amount: int)

- permanentItems: List<ShopItem>

← fixed catalog loaded once at startup never changes

- dailyOffer: DailyOffer

← refreshed automatically when current date differs from offerDate

← Shop checks this on every access not on a timer

+ listPermanentItems(): String

← returns text representation of all permanent items with prices

← called by ShopMenuController for "shop list" command

+ getDailyOffer(player: Player): String

← checks if dailyOffer.offerDate matches today refreshes if not

← player needed to pick a random plant from their unlocked plants when generating a new daily offer

← returns text representation of current daily offer

← called by ShopMenuController for "shop daily" command

+ buy(itemId: String count: int plantType: String player: Player): void

← itemId identifies what to buy

← count how many to buy

← plantType only relevant for selected seed packet purchase null for everything else

← player needed to check affordability deduct currency and add purchased items to inventory

← ShopMenuController calls this for "shop buy" command and handles the String response itself

- itemId: String

← unique identifier used in "shop buy -i <item_id>" command

- displayName: String

← shown in "shop list" output

- price: int

← cost in coins or diamonds depending on currency type

- currencyType: CurrencyType

← COINS or DIAMONDS — tells purchase logic which to deduct from Player

- quantity: int

← how many units one purchase gives

← e.g. random seed packet gives 5 selected seed packet gives 10

- maxOwnable: int

← cap on how many the player can own simultaneously

← e.g. PlantFood max is 3 Pot max is 20

← -1 means no cap

- targetPlant: PlantTemplate

← which unlocked plant's seed packets are on offer today

← randomly selected from player's unlocked plants at refresh time

- price: int

← always 1600 coins (base 2000 with 20% discount)

- quantity: int

← always 10 seed packets

- offerDate: LocalDate

← the calendar date this offer is valid for

← compared against LocalDate.now() to check if offer is still valid

- purchased: boolean

← true if player has already bought this offer today

← reset to false when offerDate changes (new day)

Player (partial — only what Shop touches)

+ getCoins(): int

+ getDiamonds(): int

+ deductCoins(amount: int): void

+ deductDiamonds(amount: int): void

+ addSeedPackets(plantName: String count: int): void

+ addPlantFood(count: int): void

+ getPlantFoodCount(): int

← to check against max 3

+ getUnlockedPots(): int

← to check against max 20

+ unlockPot(): void

+ getUnlockedPlants(): List<PlantTemplate>

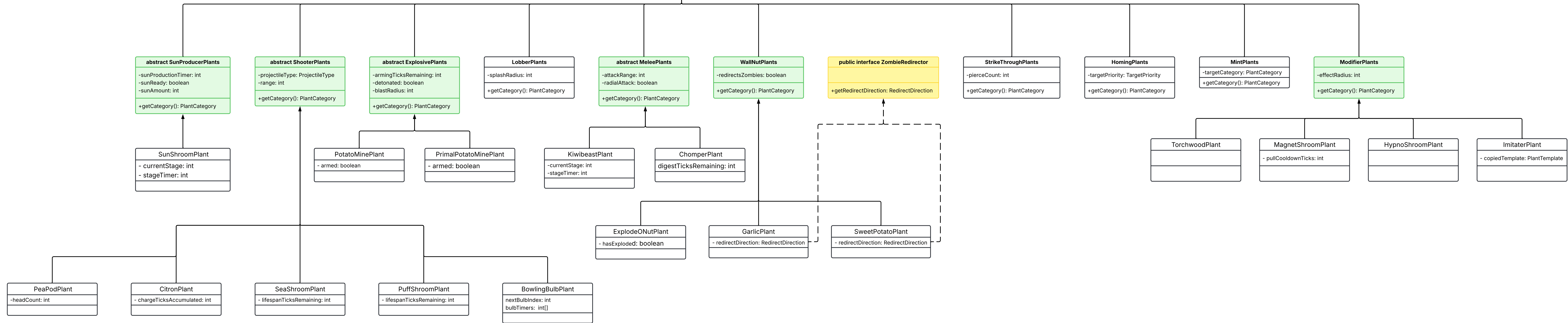
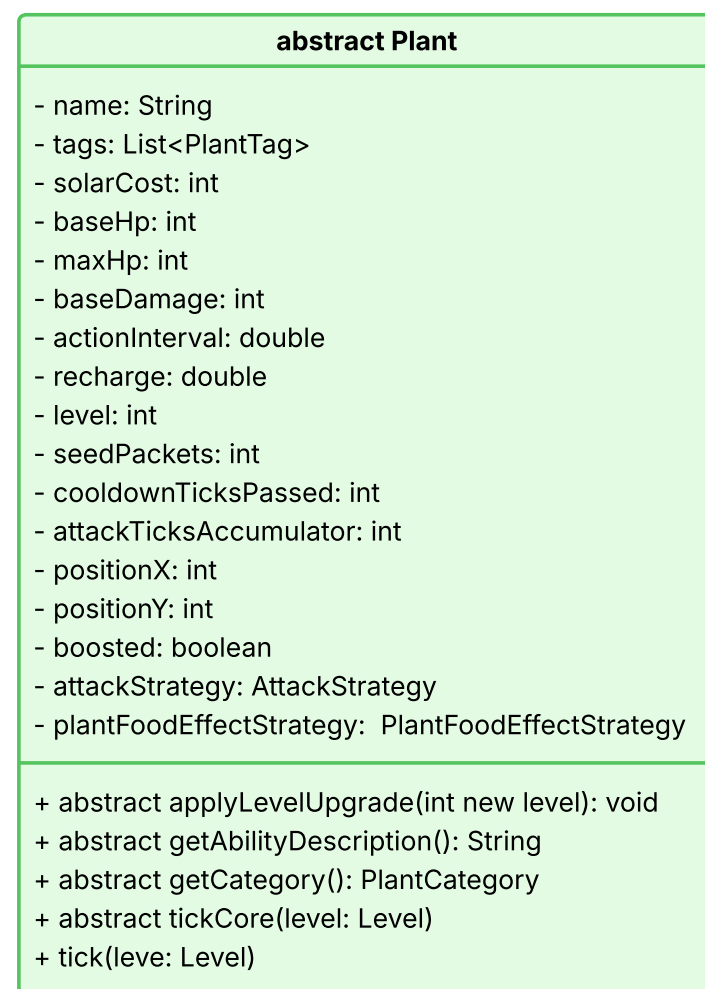
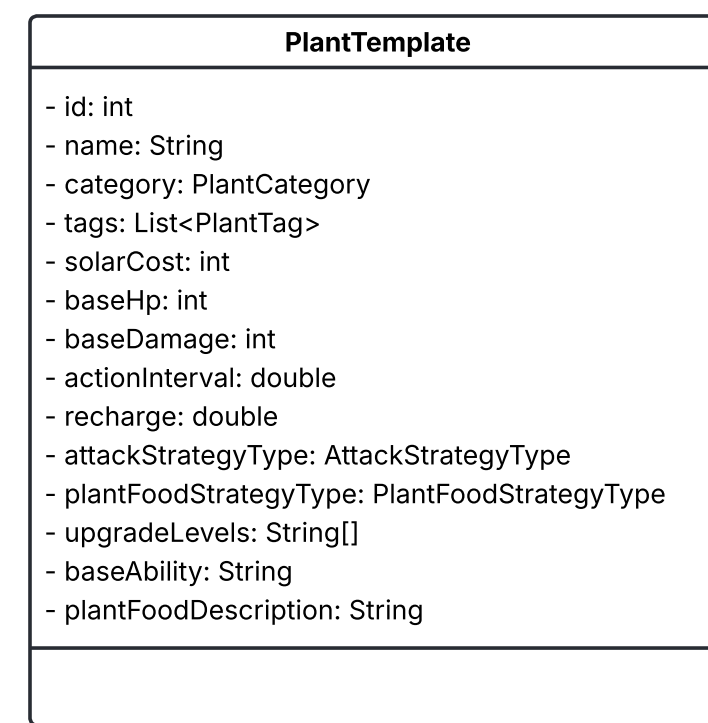
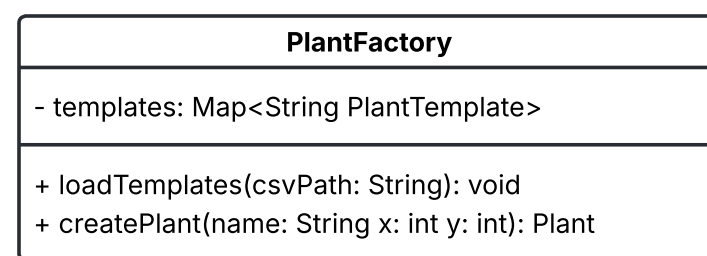
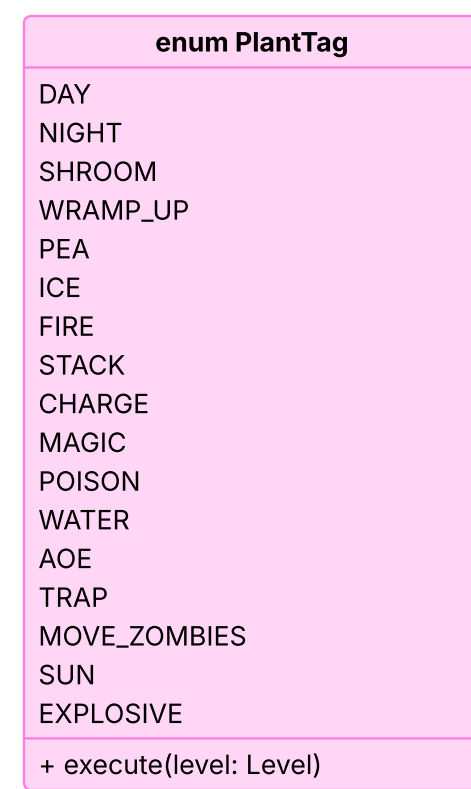
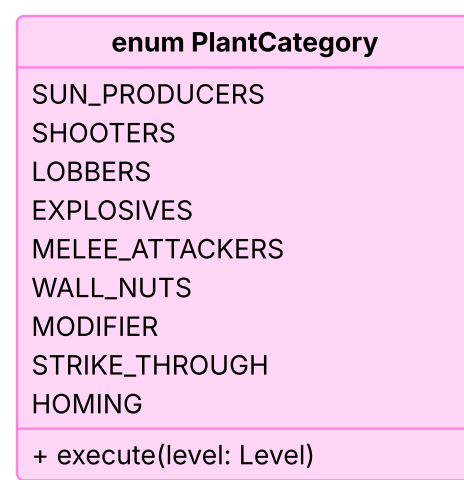
← for daily offer generation

Shop
- permanentItems: List<ShopItem> - dailyOffer: DailyOffer
+ listPermanentItems(): String + getDailyOffer(player: Player): String + buy(itemId: String count: int plantType: String player: Player): void

ShopItem
-itemId: String -displayName: String -price: int -currencyType: CurrencyType -quantity: int -maxOwnable: int

enum CurrencyType
COINS DIAMONDS

DailyOffer
-targetPlant: PlantTemplate; -price: 1600 -quantity: 10 -offerDate: LocalDate -purchased: boolean
+ operation1(params):returnType - operation2(params) - operation3()



enum CurrencyType
COINS
DIAMONDS

enum QuestCategory
ADVENTURE
SPECIAL
CHALLENGE
MYSTERY
MINIGAME

enum QuestPriority
CRITICAL
HIGH
MEDIUM
LOW

enum GameEventType
ZOMBIE_KILLED
LEVEL_COMPLETED
SUN_COLLECTED
PLANT_UPGRADED
MINIGAME_COMPLETED
PLANT_UNLOCKED
QUEST_COMPLETED

enum InventoryItemType
SEED_PACKET
PLANT_FOOD

enum RewardType
CURRENCY
UNLOCKABLE
INVENTORY

QuestReward — concrete

- rewardType: RewardType
 - ← CURRENCY UNLOCKABLE or INVENTORY
 - ← QuestManager checks this when claiming reward to decide which Player method to call
- currencyType: CurrencyType
 - ← COINS or DIAMONDS
 - ← only relevant when rewardType is CURRENCY null otherwise
 - ← reuses the same CurrencyType enum you already made in Shop
- amount: int
 - ← number of coins/diamonds when rewardType is CURRENCY
 - ← number of items when rewardType is INVENTORY
 - ← ignored when rewardType is UNLOCKABLE
- unlockTarget: String
 - ← name of plant or level to unlock
 - ← only relevant when rewardType is UNLOCKABLE null otherwise
- inventoryItemType: InventoryItemType
 - ← what kind of item to add when rewardType is INVENTORY
 - ← SEED_PACKET PLANT_FOOD etc
 - ← null when rewardType is not INVENTORY
- plantName: String
 - ← which plant the seed packets belong to
 - ← only relevant when inventoryItemType is SEED_PACKET null otherwise

QuestReward
-rewardType: RewardType
-currencyType: CurrencyType
-amount: int
-unlockTareget: String
-inventoryItemType: InventoryItemType
-plantName: String

QuestManager — concrete

- quests: List<Quest>
- ← all quests in the gameloaded at startup
- ← includes both active and completed quests
- + getQuestsForPage(category: QuestCategory): List<Quest>
- ← returns quests filtered by category,sorted by priority
- ← called by TravelMenuController when player navigates to a page
- + checkProgress(event: GameEvent,player: Player): void
- ← called by GameSessionController whenever something happens in the game — zombie killed,level completed,sun collected etc
- ← iterates relevant quests and updates their currentCount
- ← GameEvent tells QuestManager what happened so it knows which quests to update without checking all of them every tick
- + claimReward(questId: String,player: Player): void
- ← called by TravelLogMenuController when player claims a reward
- ← checks quest is completed and not already claimed
- ← applies reward to player based on reward type
- ← sets quest.claimed = true
- + refreshDailyQuests(player: Player): void
- ← checks all DailyQuest instances for expiry
- ← resets expired ones with new targets
- ← called by TravelLogMenuController when travel-log is opened

QuestManager
-quests: List<Quest>
+getQuestsForPage(category: QuestCategory): List<Quest>
+checkProgress(event: GameEvent player: Player): float
+claimReward(questId: String, player: Player): void
+ refreshDailyQuests(player: Player): void

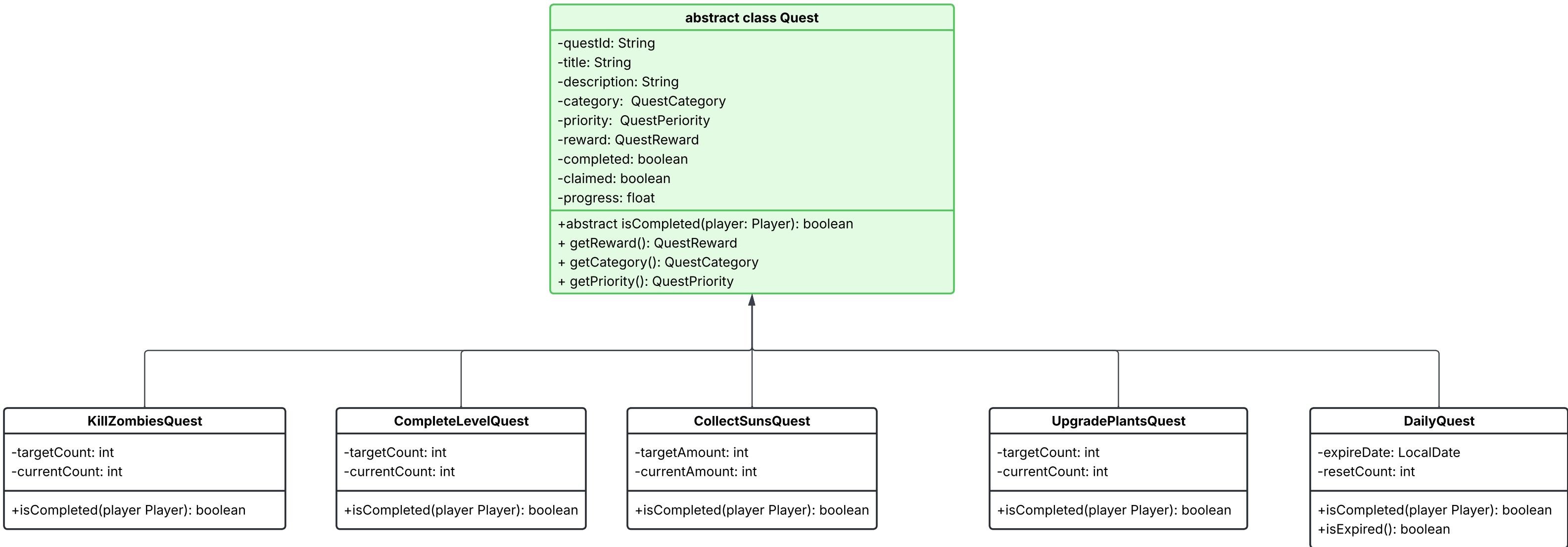
GameEvent — concrete

- eventType: GameEventType
 - ← what happened — ZOMBIE_KILLED LEVEL_COMPLETED SUN_COLLECTED PLANT_UPGRADED MINIGAME_COMPLETED etc
 - ← QuestManager switches on this to know which quests to update
- value: int
 - ← how much — e.g. how many suns collected how many zombies killed
 - ← for events where quantity matters
- player: Player
 - ← who triggered this event

GameEvent
-eventType: GameEventType
-value: int
-player: Player

Quest — abstract

- questId: String
 - ← unique identifier for this quest
- title: String
 - ← display name shown in travel-log
- description: String
 - ← explains what the player needs to do
- category: QuestCategory
 - ← ADVENTURE SPECIAL CHALLENGE or MYSTERY
 - ← determines which page in travel-log this quest appears on
- priority: QuestPriority
 - ← CRITICAL HIGH MEDIUM or LOW
 - ← used by QuestManager to sort quests within their page
- reward: QuestReward
 - ← what the player gets upon completion
- completed: boolean
 - ← true when isCompleted() returns true and reward has been claimed
- claimed: boolean
 - ← tracks whether reward was already given to player
 - ← prevents double-claiming if quest completion is checked multiple times
- + abstract isCompleted(player: Player): boolean
 - ← each quest subclass defines its own completion condition
 - ← player passed in because completion depends on player's current stats
 - ← e.g. KillZombiesQuest checks player's total zombie kill count
- + getReward(): QuestReward
- + getCategory(): QuestCategory
- + getPriority(): QuestPriority



KillZombiesQuest extends Quest — concrete

- targetCount: int
 - ← how many zombies need to be killed to complete this quest
- currentCount: int
 - ← incremented by QuestManager each time a zombie dies in GameSession
- + isCompleted(player: Player): boolean
 - ← returns true if currentCount >= targetCount

CompleteLevelsQuest extends Quest — concrete

- targetCount: int
 - ← how many levels need to be completed
- currentCount: int
- + isCompleted(player: Player): boolean

CollectSunsQuest extends Quest — concrete

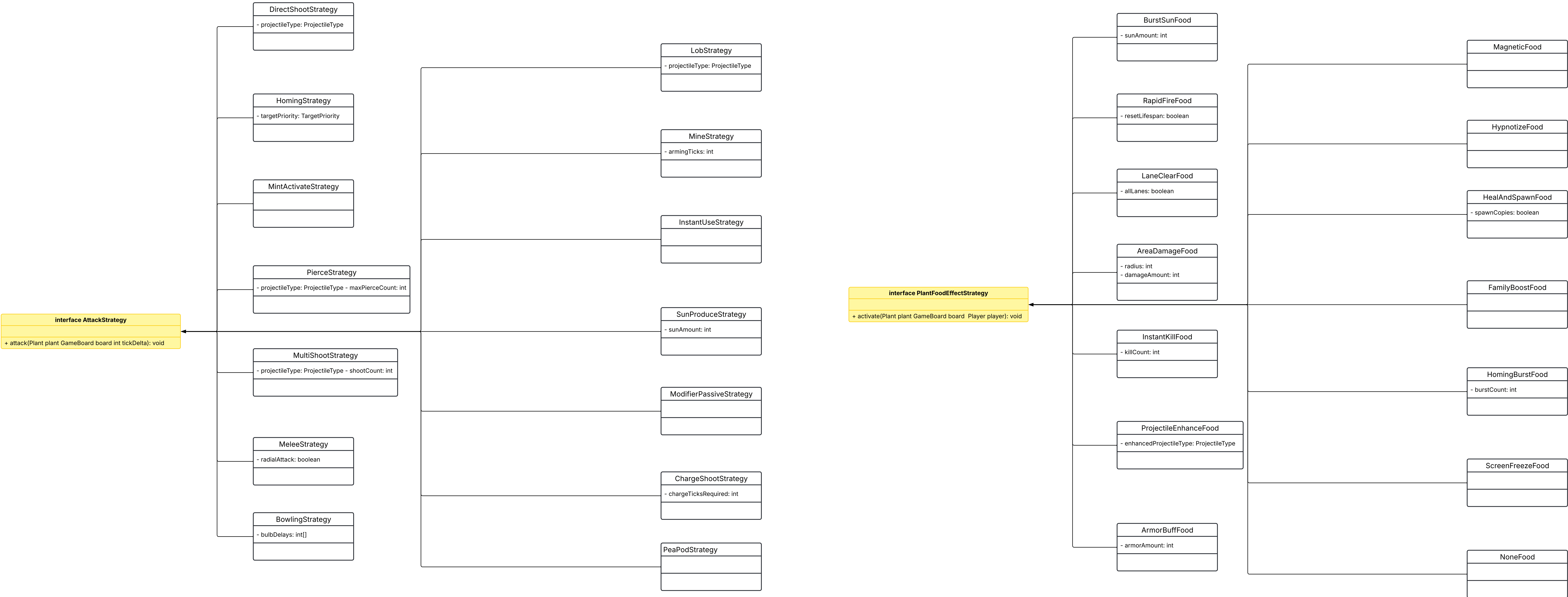
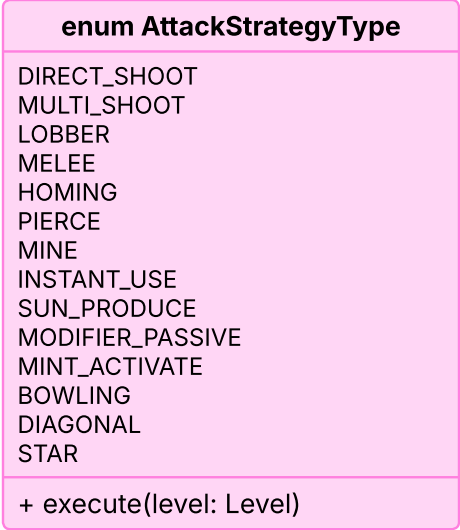
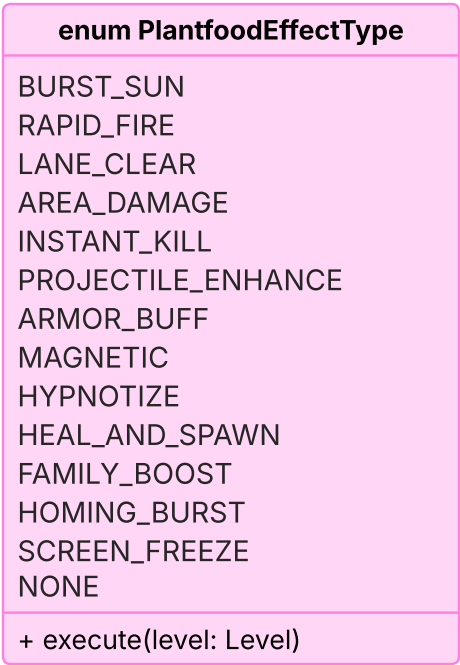
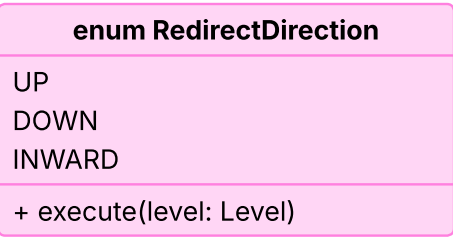
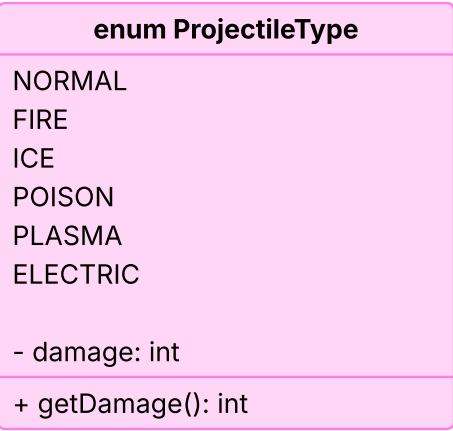
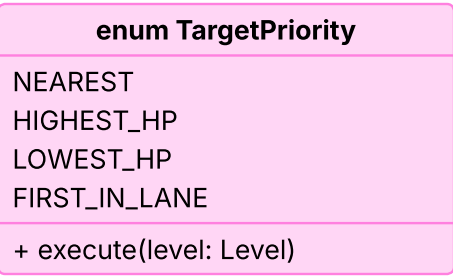
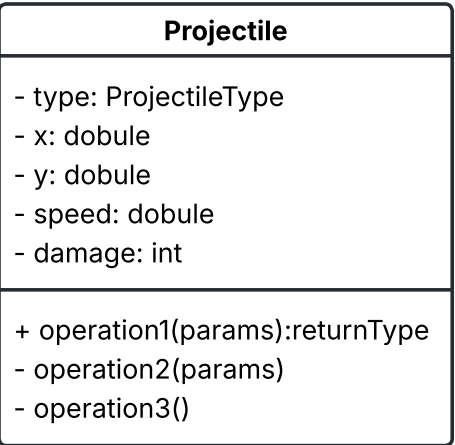
- targetAmount: int
 - ← total sun that needs to be collected across games
- currentAmount: int
- + isCompleted(player: Player): boolean

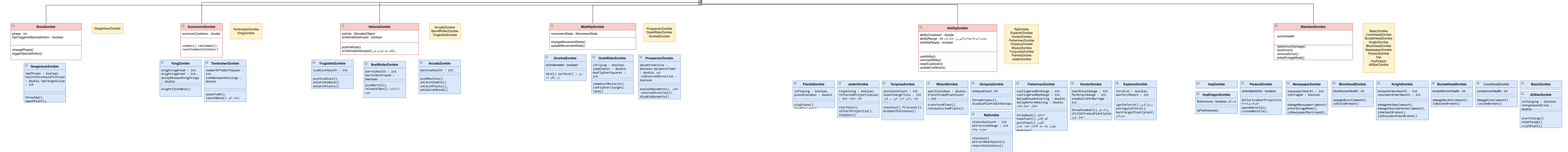
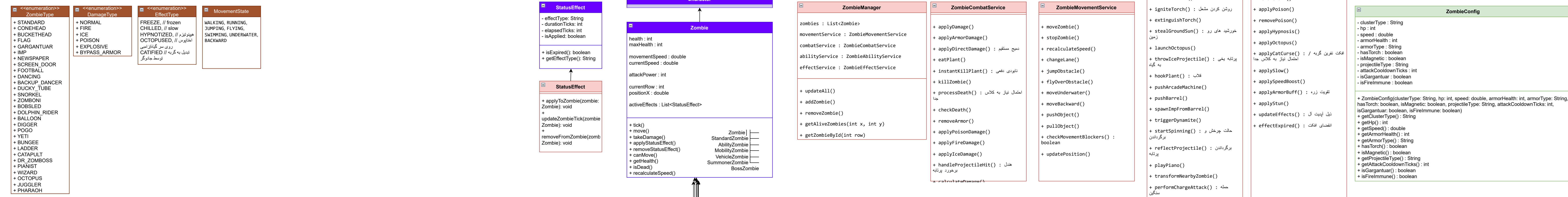
UpgradePlantsQuest extends Quest — concrete

- targetCount: int
 - ← how many plant upgrades needed
- currentCount: int
- + isCompleted(player: Player): boolean

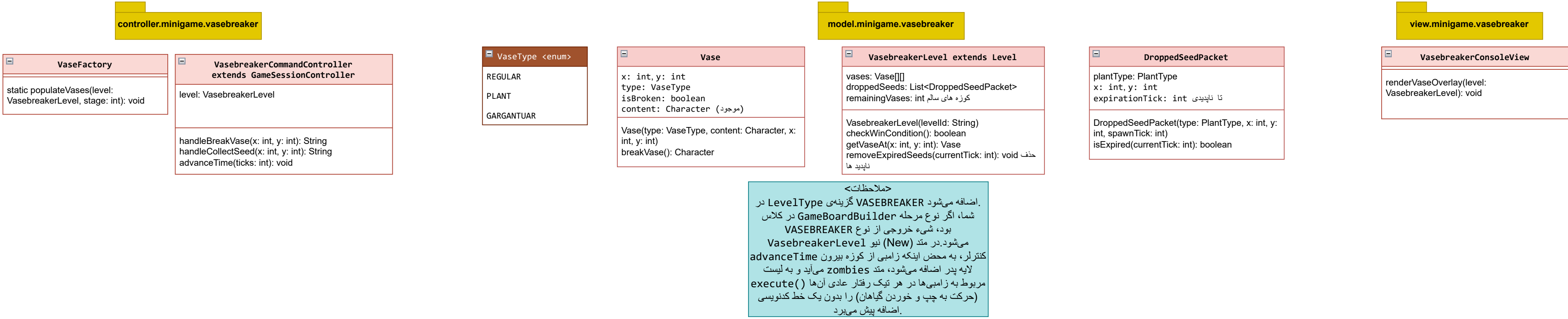
DailyQuest extends Quest — concrete

- expiryDate: LocalDate
 - ← date this daily quest expires
 - ← QuestManager checks this daily and resets expired daily quests
- resetCount: int
 - ← how many times this daily quest has been reset/refreshed
- + isCompleted(player: Player): boolean
- + isExpired(): boolean
 - ← returns true if LocalDate.now() is after expiryDate

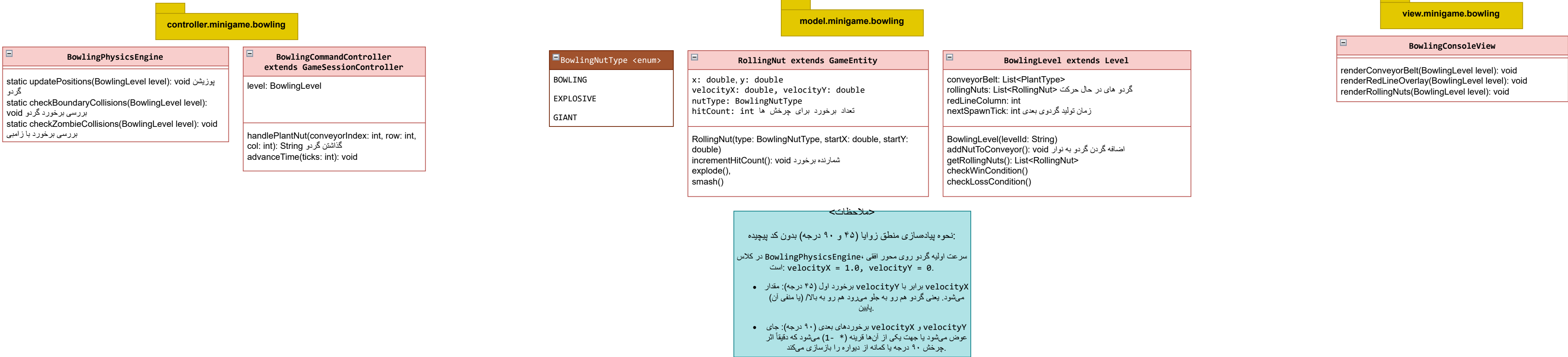




Vasebreaker



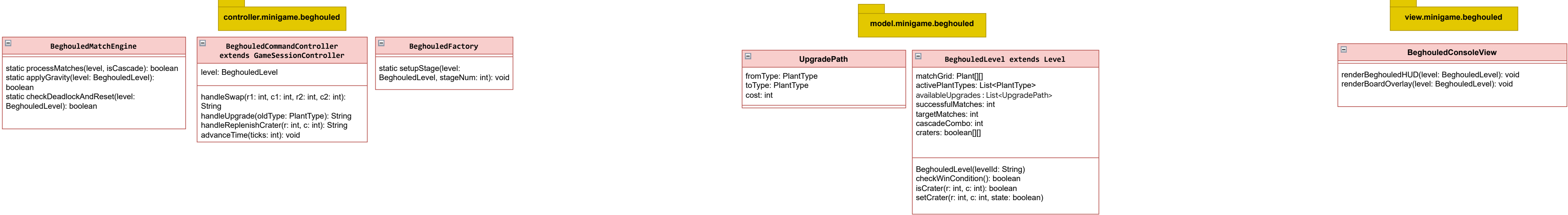
Wallnut Bowling



I Zombie



Beghouled



Zombotany

